# Práctica 1

**Caso 1 — Reservas de hotel**

```
CREATE TABLE reservas (
  id INT PRIMARY KEY,
  huesped VARCHAR(100),
  habitaciones VARCHAR(255)  -- ej: '101, 203, 305'
);
```

El error es el uso de un atributo huesped y un atributo habitaciones cuando lo ideal seria una tabla huesped y una tabla habitaciones donde se llama con el id, ya que una reserva puede tener multiples habitaciones en ella y un huesped puede tener varias reservas.

Solución:

CREATE TABLE huespedes(
huesped_id INT PRIMARY KEY,
nombre VARCHAR (50)
);

CREATE TABLE habitaciones(
habitacion_id INT PRIMARY KEY,
ubicacion VARCHAR(255)
);

CREATE TABLE reservas(
reserva_id INT PRIMARY KEY,
huesped_id INT,
FOREIGN KEY huesped_id REFERENCES huespedes(huesped_id),
);

CREATE TABLE reserva_habitaciones (
    reserva_id INT,
    habitacion_id INT,
    PRIMARY KEY (reserva_id, habitacion_id),
    FOREIGN KEY (reserva_id) REFERENCES reservas(reserva_id),
    FOREIGN KEY (habitacion_id) REFERENCES habitaciones(habitacion_id)
);

**Caso 2 — Inscripciones a talleres**

```
CREATE TABLE inscripciones (
  participante_id INT,
  taller_id INT,
  participante_nombre VARCHAR(100),
  calificacion DECIMAL(5,2),
  PRIMARY KEY (participante_id, taller_id)
);
```

El campo participante_nombre depende únicamente de participante_id, no de la llave primaria compuesta completa. Esto genera redundancia y una dependencia parcial, incumpliendo la segunda forma normal. Se puede solucionar separando los participantes de las inscripciones. Así, los datos del participante se almacenan una sola vez.

```
CREATE TABLE participantes (
    id INT PRIMARY KEY,
    nombre VARCHAR(100)
);

CREATE TABLE talleres (
    id INT PRIMARY KEY,
    nombre VARCHAR(100)
);

CREATE TABLE inscripciones (
    participante_id INT,
    taller_id INT,
    calificacion DECIMAL(5,2),
    PRIMARY KEY (participante_id, taller_id),
    FOREIGN KEY (participante_id) REFERENCES participantes(id),
    FOREIGN KEY (taller_id) REFERENCES talleres(id)
);
```

**Caso 3 — Catálogo de productos**

```
CREATE TABLE productos (
  id INT PRIMARY KEY,
  nombre VARCHAR(100),
  categoria_id INT,
  categoria_descripcion VARCHAR(200)
);
```

El campo categoria_descripcion depende de categoria_id y no directamente del producto. Esto genera una dependencia transitiva y redundancia de información, incumpliendo la tercera forma normal. Se puede solucionar creando una tabla independiente para las categorías y relacionándola con productos mediante una llave foránea.

```
CREATE TABLE categorias (
    id INT PRIMARY KEY,
    descripcion VARCHAR(200)
);

CREATE TABLE productos (
    id INT PRIMARY KEY,
    nombre VARCHAR(100),
    categoria_id INT,
    FOREIGN KEY (categoria_id) REFERENCES categorias(id)
);
```

**Caso 4 — Pedidos de una tienda**

```
CREATE TABLE pedidos (
  id INT PRIMARY KEY,
  cliente_nombre VARCHAR(100),
  cliente_correo VARCHAR(100),
  fecha DATE
);
```

Los datos del cliente (cliente_nombre y cliente_correo) se repiten en cada pedido que realiza. Esto provoca redundancia y problemas al actualizar la información del cliente. Se puede solucionar creando una tabla de clientes y relacionándola con los pedidos mediante una llave foránea.

```
CREATE TABLE clientes (
   id INT PRIMARY KEY,
   nombre VARCHAR(100),
   correo VARCHAR(100)
);

CREATE TABLE pedidos (
   id INT PRIMARY KEY,
   cliente_id INT,
   fecha DATE,
   FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);
```

**Caso 5 — Empleados de una empresa**

```
CREATE TABLE empleados (
  nombre VARCHAR(100) PRIMARY KEY,
  departamento VARCHAR(100),
  salario DECIMAL(10,2)
);
```

Se está utilizando el nombre del empleado como llave primaria. Esto es incorrecto porque dos empleados pueden tener el mismo nombre y el nombre podría cambiar.
Se puede solucionar creando un identificador único para cada empleado, utilizando un campo id como llave primaria.

```
CREATE TABLE empleados (
   id INT PRIMARY KEY,
   nombre VARCHAR(100),
   departamento VARCHAR(100),
   salario DECIMAL(10,2)
);
```

**Caso 6 — Clientes de una clínica**

```
CREATE TABLE clientes (
  id INT PRIMARY KEY,
  nombre VARCHAR(100),
  telefono INT
);
```

El campo telefono está definido como INT, pero un número telefónico no es una cantidad matemática. Puede contener ceros al inicio, códigos de país y otros caracteres. Se puede solucionar cambiando el tipo de dato a VARCHAR para almacenar correctamente los números telefónicos.

```
CREATE TABLE clientes (
    id INT PRIMARY KEY,
    nombre VARCHAR(100),
    telefono VARCHAR(20)
);
```

**Caso 7 — Cursos y estudiantes**

```
CREATE TABLE cursos (
  id INT PRIMARY KEY,
  nombre VARCHAR(100),
  estudiante_id INT  -- se supone que un curso puede tener muchos estudiantes
);
```

Se está colocando estudiante_id dentro de la tabla cursos, pero un curso puede tener muchos estudiantes. Esto dificulta registrar varios estudiantes en un mismo curso y no representa correctamente la relación muchos a muchos. Se puede solucionar creando una tabla independiente para estudiantes y una tabla intermedia para registrar las inscripciones de cada estudiante en los cursos.

```
CREATE TABLE estudiantes (
    id INT PRIMARY KEY,
    nombre VARCHAR(100)
);

CREATE TABLE cursos (
    id INT PRIMARY KEY,
    nombre VARCHAR(100)
);

CREATE TABLE inscripciones (
    estudiante_id INT,
    curso_id INT,
    PRIMARY KEY (estudiante_id, curso_id),
    FOREIGN KEY (estudiante_id) REFERENCES estudiantes(id),
    FOREIGN KEY (curso_id) REFERENCES cursos(id)
);
```

**Caso 8 — Facturas de una empresa**

```
CREATE TABLE facturas (
  id INT PRIMARY KEY,
  cliente_id INT,
  monto DECIMAL(10,2)
);
```

El campo monto permite valores nulos y negativos porque no tiene restricciones que validen la información. Esto puede provocar facturas sin monto o con cantidades inválidas. Se

puede solucionar agregando la restricción NOT NULL para obligar a registrar un monto y CHECK para impedir valores negativos.

```
CREATE TABLE facturas (
    id INT PRIMARY KEY,
    cliente_id INT,
    monto DECIMAL(10,2) NOT NULL,
    CHECK (monto >= 0)
);
```

# Práctica 2

**Error 1. Tabla miembros — Tipo de dato incorrecto**
Problema: El atributo telefono está definido como INT. Un teléfono es un dato de identificación/contacto, no una cantidad para realizar operaciones matemáticas. INT puede perder ceros iniciales y no representa adecuadamente prefijos o símbolos.
Corrección: Cambiar telefono a VARCHAR(20). Los teléfonos de ejemplo (88881111, 88882222 y 88883333) se conservan como cadenas de dígitos.

**Error 2. Tabla clases — Información redundante**
Problema: instructor_nombre repite el nombre que ya se almacena en instructores. Si el nombre cambia, ambas tablas podrían quedar con valores diferentes.
Corrección: Eliminar instructor_nombre y obtener el nombre mediante instructor_id con un JOIN. La relación instructor_id ya existe y los registros de ejemplo conservan sus identificadores.

**Error 3. Tabla inscripciones_clases — Falta de llave foránea**
Problema: Sin una llave foránea, se puede insertar una inscripción cuyo miembro no exista en miembros. Esto deja referencias huérfanas y afecta la confiabilidad de los reportes.
Corrección: Agregar una llave foránea desde inscripciones_clases.miembro_id hacia miembros.id. Los valores de ejemplo 1, 2 y 3 existen en miembros.

**Error 4. Tabla inscripciones_clases — Falta de llave foránea**
Problema: Sin una llave foránea, se puede registrar una inscripción para una clase inexistente. Esto produce datos que no se pueden asociar con una clase real.
Corrección: Agregar una llave foránea desde inscripciones_clases.clase_id hacia clases.id. Los valores de ejemplo 1 y 2 existen en clases.

**Error 5. Tabla pagos — Falta de relación con membresias**
Problema: El campo membresia contiene el tipo de plan, pero no está relacionado con membresias.tipo. Se podrían registrar pagos con tipos de membresía que no están definidos en el catálogo.
Corrección: Agregar una llave foránea desde pagos.membresia hacia membresias.tipo. Los valores Mensual y Trimestral existen en membresias.

**Error 6. Tabla pagos — Falta de restricciones en el monto**
Problema: monto acepta NULL y valores negativos. Un pago sin monto o con un importe negativo puede distorsionar los registros contables y los reportes de ingresos.
Corrección: Modificar monto para que sea NOT NULL y agregar CHECK (monto >= 0). Los pagos de ejemplo tienen montos positivos (25000 y 65000), por lo que siguen siendo válidos.

Query 1

```
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107   -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| tipo | varchar(50) | NO | PRI | NULL | |
| precio | decimal(10,2) | NO | | NULL | |
| duracion_meses | int | NO | | NULL | |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

Query 1

```
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107   -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| id | int | NO | PRI | NULL | auto_increment |
| nombre | varchar(100) | NO | | NULL | |
| telefono | varchar(20) | YES | | NULL | |
| correo | varchar(100) | YES | | NULL | |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

Query 1

```
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107   -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| id | int | NO | PRI | NULL | auto_increment |
| nombre | varchar(100) | YES | | NULL | |
| especialidad | varchar(100) | YES | | NULL | |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

Query 1

```
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107   -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| id | int | NO | PRI | NULL | auto_increment |
| nombre | varchar(100) | NO | | NULL | |
| instructor_id | int | YES | MUL | NULL | |
| horario | time | YES | | NULL | |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

Query 1

```
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107   -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| id | int | NO | PRI | NULL | auto_increment |
| miembro_id | int | YES | MUL | NULL | |
| clase_id | int | YES | MUL | NULL | |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

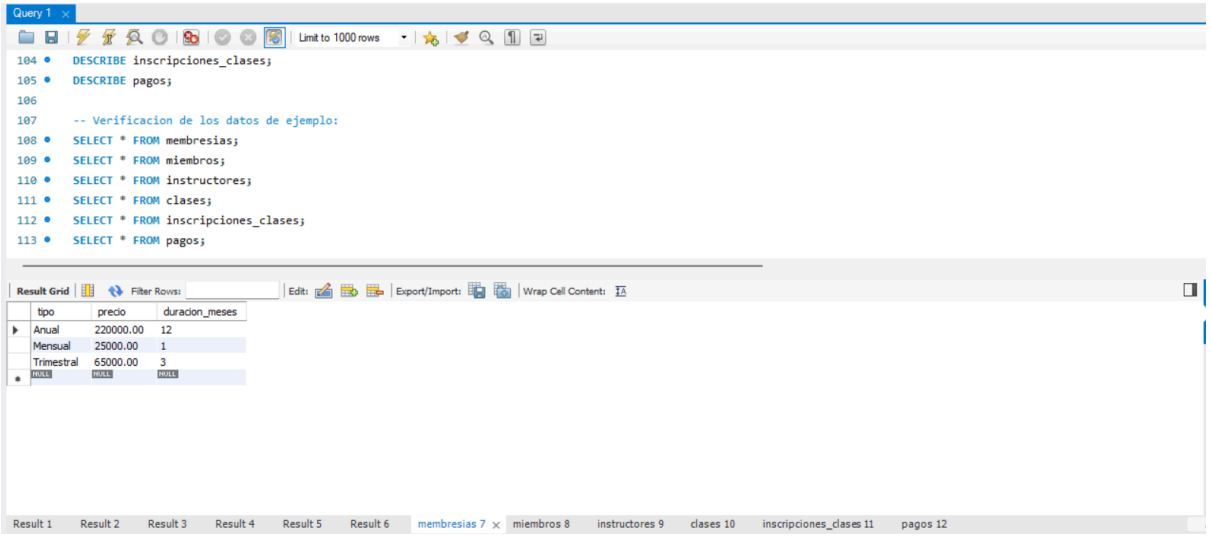
Query 1

```
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107   -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| id | int | NO | PRI | NULL | auto_increment |
| miembro_id | int | YES | MUL | NULL | |
| membresia | varchar(50) | YES | MUL | NULL | |
| monto | decimal(10,2) | NO | | NULL | |
| fecha | date | YES | | NULL | |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

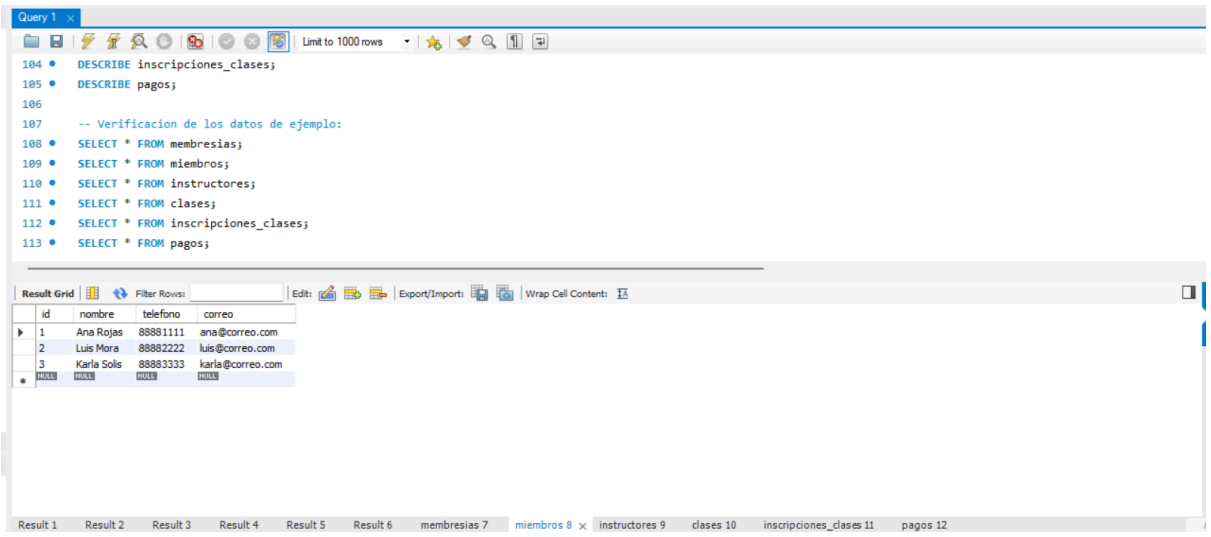


```
Query 1
Limit to 1000 rows
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107     -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| tipo | precio | duracion_meses |
|---|---|---|
| Anual | 220000.00 | 12 |
| Mensual | 25000.00 | 1 |
| Trimestral | 65000.00 | 3 |
| NULL | NULL | NULL |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

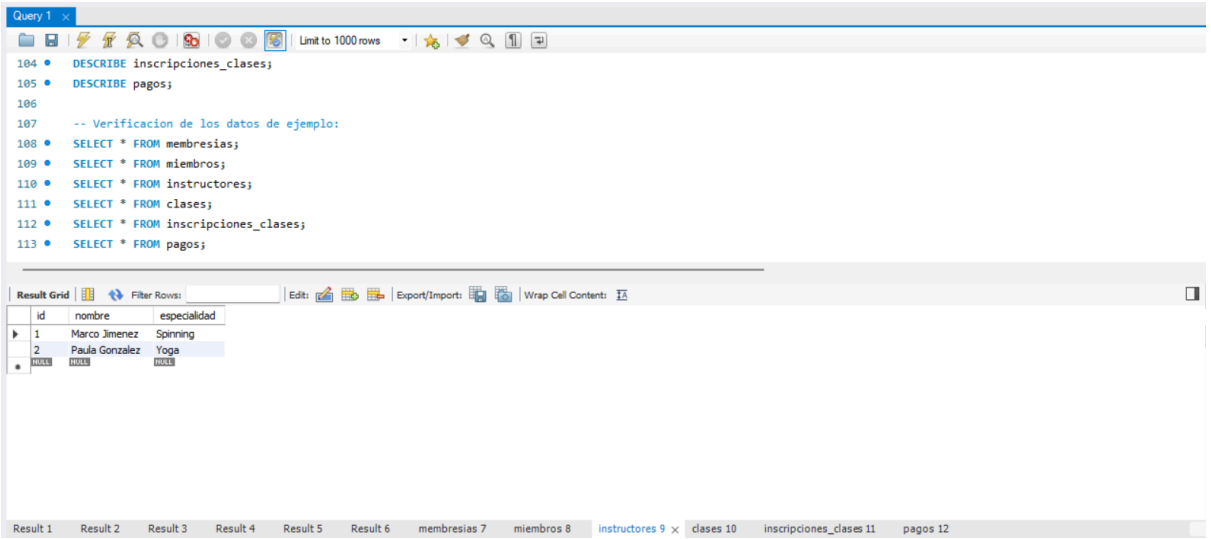


```
Query 1
Limit to 1000 rows
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107     -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| id | nombre | telefono | correo |
|---|---|---|---|
| 1 | Ana Rojas | 88881111 | ana@correo.com |
| 2 | Luis Mora | 88882222 | luis@correo.com |
| 3 | Karla Solis | 88883333 | karla@correo.com |
| NULL | NULL | NULL | NULL |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12



```
Query 1
Limit to 1000 rows
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107     -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| id | nombre | especialidad |
|---|---|---|
| 1 | Marco Jimenez | Spinning |
| 2 | Paula Gonzalez | Yoga |
| NULL | NULL | NULL |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

Query 1

```
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107     -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid

| id | nombre | instructor_id | horario |
|---|---|---|---|
| 1 | Spinning matutino | 1 | 06:00:00 |
| 2 | Yoga vespertino | 2 | 18:00:00 |
| NULL | NULL | NULL | NULL |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

Query 1

```
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107     -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid

| id | miembro_id | clase_id |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 2 | 1 |
| 3 | 3 | 2 |
| NULL | NULL | NULL |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12

Query 1

```
104 • DESCRIBE inscripciones_clases;
105 • DESCRIBE pagos;
106
107     -- Verificacion de los datos de ejemplo:
108 • SELECT * FROM membresias;
109 • SELECT * FROM miembros;
110 • SELECT * FROM instructores;
111 • SELECT * FROM clases;
112 • SELECT * FROM inscripciones_clases;
113 • SELECT * FROM pagos;
```

Result Grid

| id | miembro_id | membresia | monto | fecha |
|---|---|---|---|---|
| 1 | 1 | Mensual | 25000.00 | 2026-09-01 |
| 2 | 2 | Trimestral | 65000.00 | 2026-09-01 |
| NULL | NULL | NULL | NULL | NULL |

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | membresias 7 | miembros 8 | instructores 9 | clases 10 | inscripciones_clases 11 | pagos 12